Object-Oriented Design

Making Design Decisions Explicit

FinTech 512

Duke

Object-Oriented Design: Making Design Decisions Explicit

- Software systems are collections of **decisions**
- Object-oriented design is a way to:
  - group related decisions
  - assign clear responsibilities
  - control how parts of a system interact
- OO helps make design visible, discussable, and reviewable

Objects as Responsibility Holders

In financial systems, objects often represent things that:

- **Own rules**
  FeeCalculator, RiskPolicy, RefundPolicy
- **Enforce invariants**
  AccountBalance, Ledger, Authorization
- **Coordinate workflows**
  PaymentProcessor, SettlementService

Objects exist to own behavior — not just data.

Duke

Object Collaboration Over Inheritance

- Objects **collaborate** to deliver system behavior
- Most OO design is about:
  - delegation
  - composition
  - clear interfaces
- Inheritance is a tool — **not the default**

OO Vocabulary Alignment

- Everything is an object
- Program is a set of objects sending messages
- Objects have state (memory)
- Objects have a type
- Objects of the same type respond to the same messages

Object: state, behavior, and identity

We care about objects because they **own behavior and decisions** as well as **hold data**.

Classes as Responsibility Containers

Class

- A class describes a **type of object** we might create
- Defines the object's **responsibilities**
  - what it *knows* (state)
  - what it *does* (behavior)

Classes in FinTech Systems Often Represent:

- **Policies** → FeeCalculator, RiskPolicy
- **Constraints & invariants** → Ledger
- **Domain aggregates** → Portfolio
- **Domain entities** → Stock, Transaction

Behavior (Methods)

- Actions the object is **responsible for**
  - apply_discounts()
  - rebalance()
  - record_entry()

State (Fields)

- Data the object needs to **fulfill its responsibilities**
- Example:
  - A Portfolio has holdings and allocations
  - A Ledger has accounts and entries

Duke

Two Kinds of Objects in a Trading System

In a stock trading system, objects usually fall into two broad roles:

- **Entities**
  Represent facts and events
  What happened

- **Policies**
  Represent rules and decisions
  What should happen

Confusing these roles leads to brittle systems.

Duke

Entities: What Happened

Entities capture **events and state** in the system.

- Stock
- Trade
- Transaction
- Account

Typical responsibilities:

- hold data
- enforce basic invariants
- provide derived information
- Entities describe reality — they don’t decide policy.

Duke

Policies: What Should Happen

Policies encode **rules, constraints, and decisions**.

- RiskPolicy
- FeeCalculator
- TradingRules
- CompliancePolicy

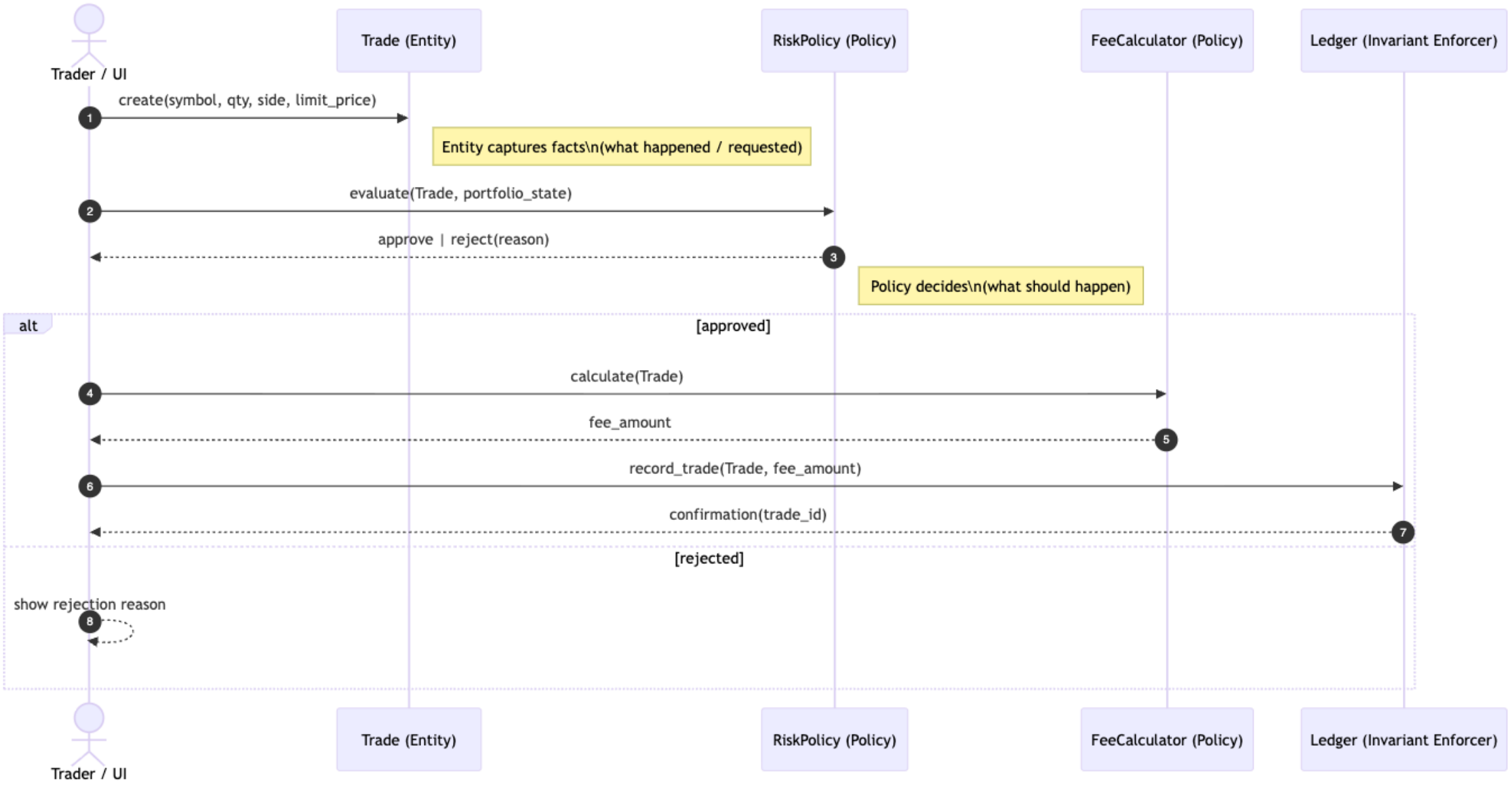
Typical responsibilities:

- evaluate conditions
- enforce limits
- approve or reject actions
- explain decisions

Policies decide — they don’t store history.

Duke

Trade Execution: Who Does What?



When Entities Start Making Decisions

Warning signs:

- conditionals inside entities
- fee logic in Transaction
- risk checks in Stock
- policy rules scattered across classes

```
# Bad: Entity making policy decisions
class Transaction:
    def calculate_fee(self):
        if self.amount > 10000:
            return self.amount * 0.001
        return self.amount * 0.002
```

This makes systems harder to change and review.

Duke

HAS-A vs IS-A: Modeling Relationships

IS-A → specialization
Inheritance ("is a kind of")

HAS-A → composition
Ownership or collaboration ("has a")

Most design decisions are about **HAS-A**, not **IS-A**.

Duke

IS-A (Inheritance): Specialization

Examples (Trading System):

- LimitOrder **is a** Order
- MarketOrder **is a** Order

Characteristics

- Strong coupling
- Shared interface and behavior
- Changes ripple through hierarchy

Use IS-A only when substitutability truly holds.

Duke

HAS-A (Composition): Ownership & Collaboration

Examples (Trading System):

- Portfolio **has a** collection of Stock holdings
- TradeExecutor **has a** RiskPolicy
- TradeExecutor **has a** FeeCalculator
- Ledger **has a** list of entries

Characteristics

- Contains or owns
- Uses to fulfill responsibility
- Lifecycle awareness
- Replaceable
- Limits change impact

HAS-A relationships localize change.

Duke

When IS-A Is Misused

Warning Signs

- Inheritance used to reuse code
- Subclasses override large portions of behavior
- Business rules leak into base classes

Bad Example

- RiskAwareTrade **is a** Trade

Inheritance for convenience usually creates hidden coupling.

Choosing HAS-A VS IS-A

For each of these choose the right relationship (has-a, is-a, NA):

- FileHandler ____ File
- TextIOWrapper ____ BufferedIOBase
- datetime.datetime ____ datetime.date
- LimitOrder ____ Order
- MarketOrder ____ Order
- Portfolio ____ StockHoldings
- TradeExecutor ____ RiskPolicy
- Trade ____ Stock
- Ledger ____ LedgerEntries
- FeeCalculator ____ DiscountPolicy
- RiskCheckedTrade ____ Trade
- Stock ____ PriceQuote

Choosing HAS-A VS IS-A

For each of these choose the right relationship (has-a, is-a, NA):

- FileHandler **has-a (uses a)** File
- TextIOWrapper **is-a** BufferedIOBase
- datetime.datetime **is-a** datetime.date
- LimitOrder **is-a** Order
- MarketOrder **is-a** Order
- Portfolio **has-a (contains)** StockHoldings
- TradeExecutor **has-a** RiskPolicy
- Trade **has-a** Stock
- Ledger **has-a (contains)** LedgerEntries
- FeeCalculator **has-a (uses)** DiscountPolicy
- RiskCheckedTrade **NA** Trade
- Stock **NA** PriceQuote

Duke

Design Heuristics: Finding classes

Coad and Yourdon

Structure	“Is a” and “part of” relationships
Other systems	External systems with which the application interacts
Devices	Devices with which the application interacts
Events remembered	A historical event that must be recorded
Roles played	The different roles users play in interacting with the application
Locations	Physical locations, offices, and sites important to the application
Organizational units	Groups to which users belong

Coad, P., and Yourdon, E. 1990. Object-Oriented Analysis. Englewood Cliffs, NJ: Prentice-Hall

Design Heuristics: Finding Classes

Ross

People	Humans who carry out some function
Places	Areas set aside for people or things
Things	Physical objects, or groups of objects, that are tangible
Organizations	Formally organized collections of people, resources, facilities, and capabilities having a defined mission, whose existence is largely independent of individuals
Concepts	Principles or ideas not tangible per se; used to organize or keep track of business activities and/or communications
Events	Things that happen, usually to something else at a given date and time, or as steps in an ordered sequence

Ross, R. 1987. Entity Modeling: Techniques and Application. Boston, MA: Database Research Group, p. 9

Design Heuristics: Finding Classes

Shlaer and Mellor

Tangible things	Cars, telemetry data, pressure sensors
Roles	Mother, teacher, politician
Events	Landing, interrupt, request
Interactions	Loan, meeting, intersection

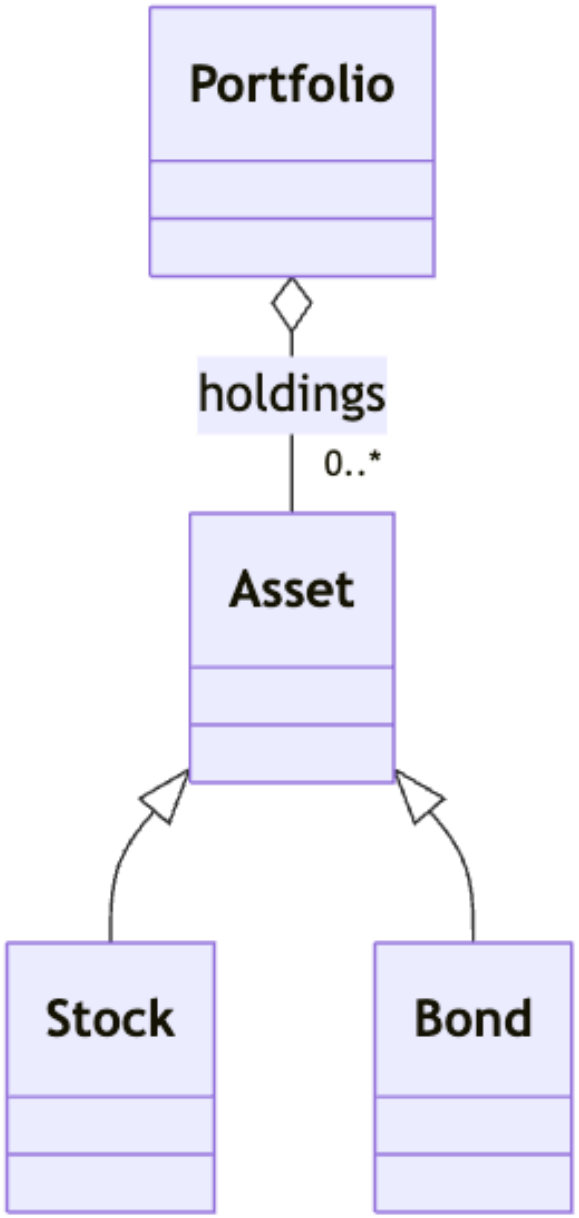
Object-Oriented Systems Analysis: Modeling the World in Data. Englewood Cliffs, NJ

Design Heuristics: Design with Objects

1. **Name the key responsibilities**
   What decisions must the system make?
2. **Assign responsibilities to objects**
   Who owns rules, invariants, or workflows?
3. **Define collaborations**
   Which objects must talk to which others?
4. **Choose boundaries and visibility**
   What should be hidden vs exposed?
5. **Define the public interface (the contract)**
   What can other objects depend on safely?

Class Diagrams: Making Design Decisions Visible

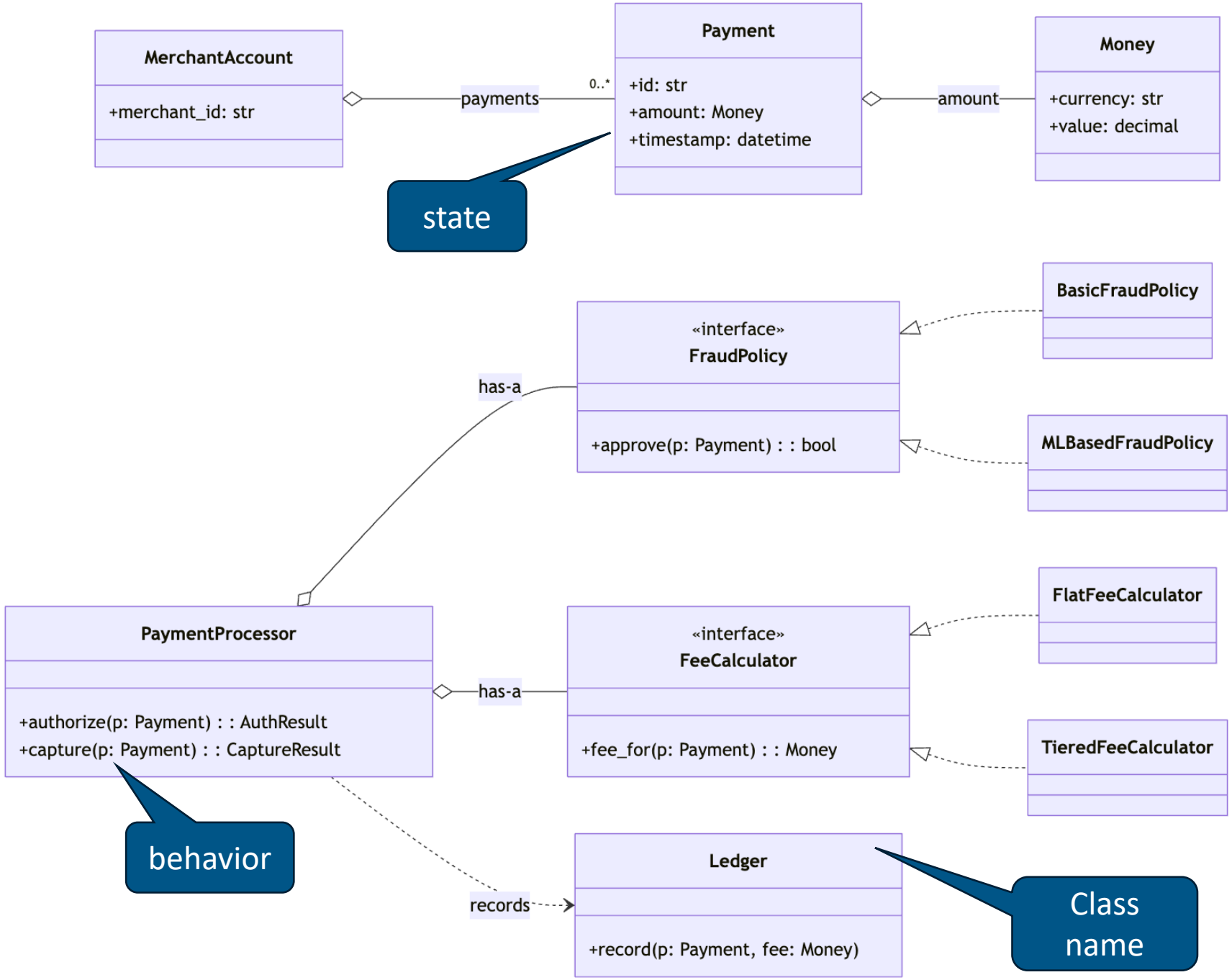
- Show **responsibilities, ownership,** and **relationships**
- Make assumptions and boundaries explicit
- Support discussion, review, and evolution
- Moved away from implementation blueprints.



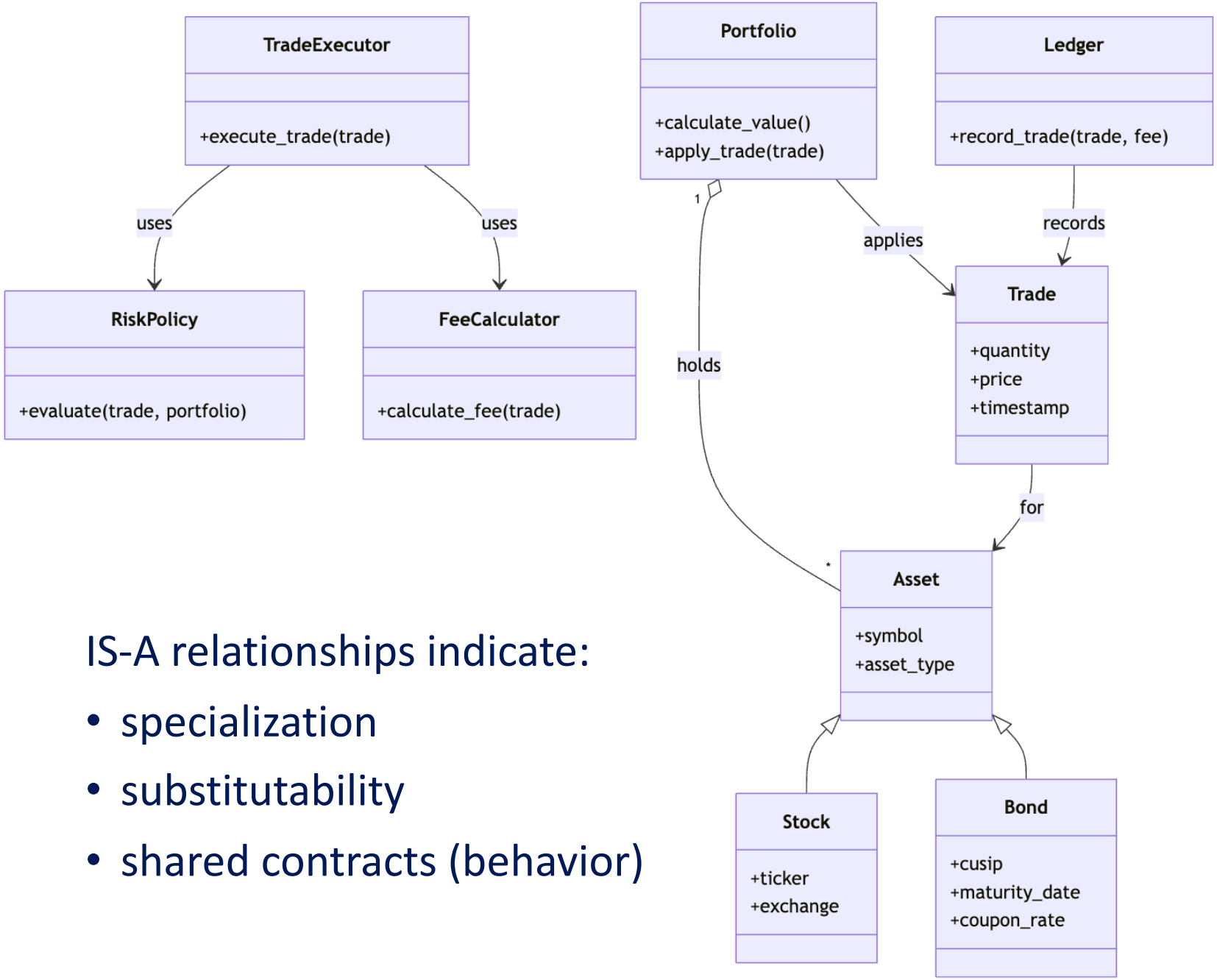
What We Care About in Class Diagrams

We use class diagrams to show:

- Key classes (responsibility holders)
- HAS-A relationships (collaboration / ownership)
- IS-A relationships (true specialization)
- Public interfaces (what others depend on)



Example: Core Trading System Classes



HAS-A relationships indicate:

- collaboration
- ownership or dependency
- replaceable behavior

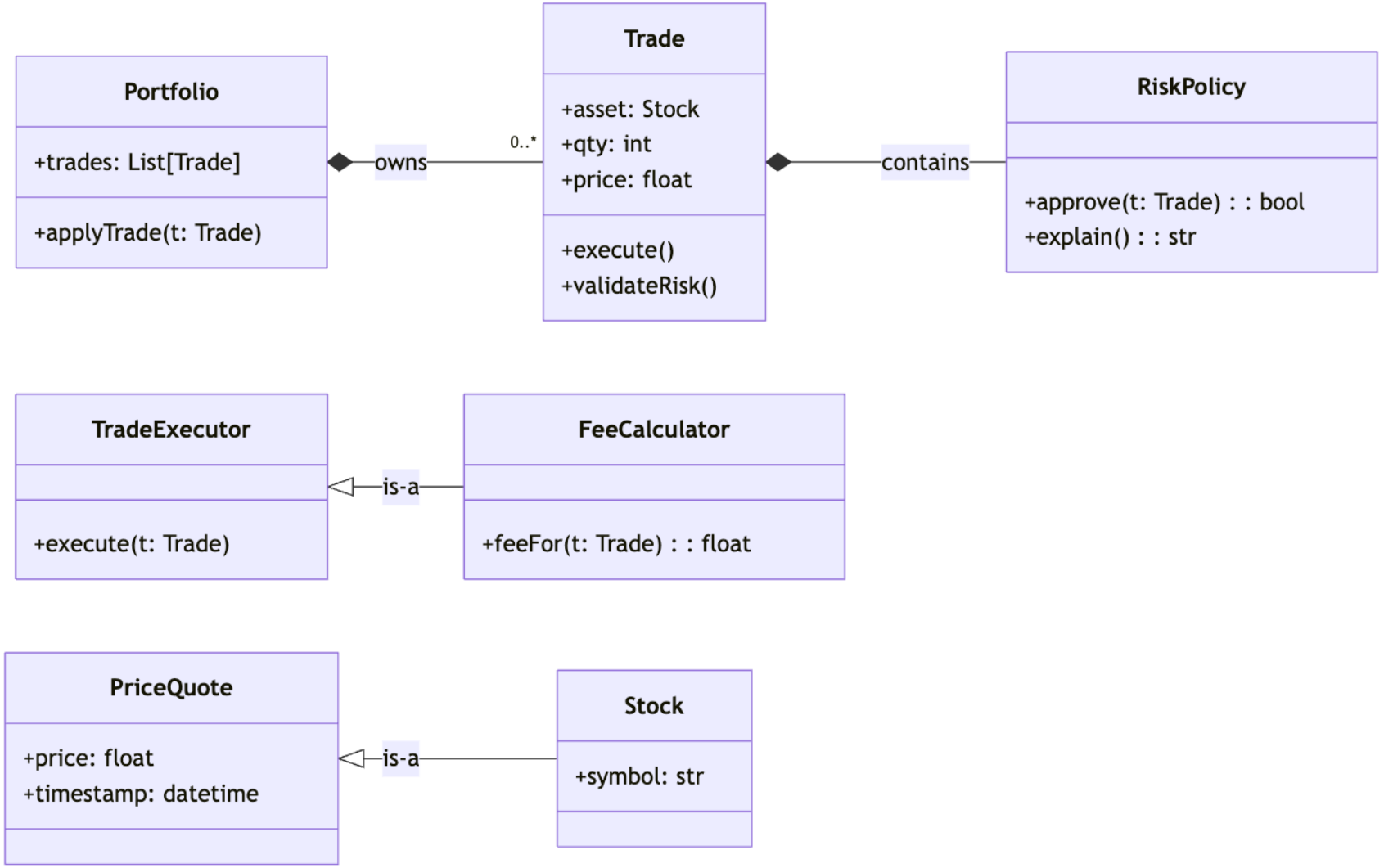
IS-A relationships indicate:

- specialization
- substitutability
- shared contracts (behavior)

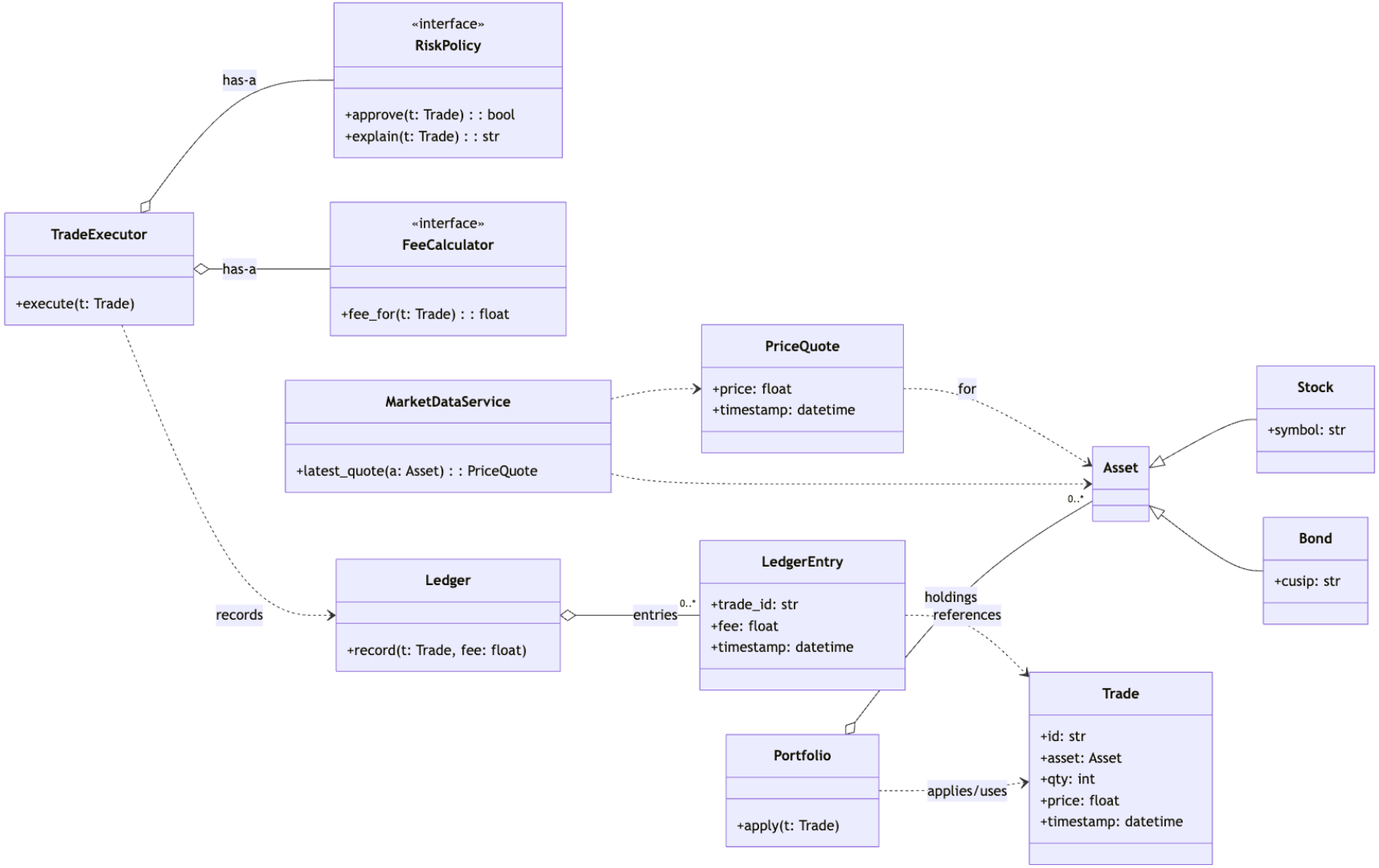
Exercise: Design Review

Consider these questions:

- What decisions are being made implicitly here?
- Where are responsibilities mixed?
- What change would be hardest to make?
- Which relationships should be HAS-A vs IS-A?



Exercise: Corrected Diagram



Interfaces & Abstract Classes (Design View)

Why we use them

- To **define boundaries** between parts of the system
- To make **replaceable decisions explicit**
- To limit what other objects are allowed to depend on

How to think about them

- An interface = *"what others can rely on"*
- An abstract class = *"shared responsibility, not shared code"*

When they help

- Policies that may change *(RiskPolicy, FeeCalculator)*
- Behavior you want to **swap or test independently**

Duke

OO Design and Testing Go Hand-in-Hand

Clear responsibilities → focused tests

- Entities → invariant tests
- Policies → decision tests
- Coordinators → interaction tests

HAS-A relationships → easy isolation

- Replace collaborators with fakes or stubs
- Test behavior without complex setup

Interfaces → controlled substitution

- Swap RiskPolicy implementations
- Test "what happens if risk rejects a trade?"

Warning Signs (Design Smells Seen Through Tests)

- Tests require large object graphs
- Mocks everywhere just to get things to run
- You can't test a policy without creating a whole system
- Tests break when unrelated behavior changes
- These are usually **design problems**, not testing problems.

This section will make more sense when we revisit testing in later lectures

Good object-oriented design makes **testing simpler, smaller, and more meaningful**.

Object-Oriented Design — Key Takeaways

What OO Design Gives Us

- A way to **group related decisions**
- Clear **ownership of responsibilities**
- Controlled **collaboration between parts**

Core Relationship Decisions

- **HAS-A (composition)** is the default
  → collaboration, ownership, replaceability, localized change
- **IS-A (inheritance)** is rare and deliberate
  → true specialization where substitutability holds
- *"If you're using inheritance to reuse code, reconsider."*

How We Use Objects in Systems

- **Objects own behavior and decisions**, not just data
  - **Entities** → facts & events (Trade, Stock, Transaction)
  - **Policies** → rules & decisions (RiskPolicy, FeeCalculator)
  - **Aggregates / Coordinators** → workflows & state views (Portfolio, TradeExecutor)
  - **Invariant Enforcers** → correctness & auditability (Ledger)

Entities describe what happened.
Policies decide what should happen.

Duke